

Distributed Rate Limiting: Centralized vs Local Approach

Repo:

<https://github.khoury.northeastern.edu/tingashley/rate-limiting>

Team Member

- Ashley Li li.ting4@northeastern.edu
- Robyn Jiang jiang.yiqi@northeastern.edu
- Yuxin Hu hu.yuxin3@northeastern.edu
- Andy Sung sung.di@northeastern.edu

Problem

In large-scale **E-commerce platforms**, thousands of requests per second come from users browsing, adding items to carts, or checking out. Without proper rate limiting, the backend can be overloaded, causing **downtime, unfair resource usage, or even financial loss**.

Our project focuses on building a **distributed rate-limiting system** for an **E-commerce backend**, ensuring users cannot exceed a fixed number of requests per time window, even when requests are distributed across multiple servers (e.g., browsing products, adding to cart, checkout APIs). We compare **centralized (Redis-based shared counter)** and **distributed (local token bucket)** approaches, evaluating trade-offs among **accuracy, latency, and fault tolerance** under real E-commerce workloads.

Why It Is Important for Future Stakeholders

This project provides significant value for future stakeholders such as E-commerce platform operators, system architects, and API developers. During high-traffic events like holiday sales, flash deals, or large-scale promotions, millions of concurrent requests can overload backend systems.

A well-designed **rate limiter** ensures:

- Fair resource usage so all users have equal access opportunities.
- Stable performance by preventing server overloads and cascading failures.
- Enhanced customer satisfaction through consistent response times.
- Operational resilience by maintaining functionality even when components like Redis fail.

For engineering and business stakeholders, this system provides data-driven insights to choose the optimal rate-limiting strategy (centralized or distributed), balance accuracy vs. latency, and build fault-tolerant architectures that can scale sustainably as traffic grows.

Core Challenge and Idea

The core challenge: ensuring a user doesn't exceed N requests per time window when their requests are distributed across multiple backend servers.

Core Idea: Compare centralized (Redis-based shared counter) versus distributed (local token bucket per server) approaches, measuring the trade-offs between accuracy, latency, and fault tolerance.

Three Main Experiments

1. **Performance Comparison** - Measure latency overhead of rate limit checking: centralized Redis (2-5ms overhead, exact limit) vs distributed local counters (0.01ms overhead, approximate limit). Test under increasing load: 100 to 10,000 requests/second.
2. **Consistency Under Load** - Test accuracy with 5 servers and different traffic distributions. Centralized should enforce exactly 100 requests/minute; distributed will allow approximately 100-150. Measure variance under uniform vs skewed load distribution.
3. **Failure Scenarios** - Test resilience: What happens when Redis fails (centralized) vs when traffic unevenly distributes across servers (distributed)? Measure recovery time and user impact during failures.

Optional but good to: Implement **hybrid approach** using local buckets with periodic Redis synchronization, comparing it to pure centralized and pure distributed strategies.

Project Plan

Week #	Goal	Status
1	Centralized Redis implementation	In process
2	Distributed local bucket	Planned
3	Three Main Experiments	Planned
4	Report and presentation	Planned

Team Role

Members #	Role	Responsibility
Robyn Jiang	Backend	Build Redis-based limiter, write report
Ashley Li	Backend	Build Redis-based limiter, write report
Yuxin Hu	Backend	Build in-memory limiter, write report
Andy Sung	Backend	Build in-memory limiter, write report

Objectives

Short-Term Objectives

- Build and benchmark centralized and distributed rate limiters.
- Measure latency, consistency, and recovery performance.
- Integrate system into an E-commerce microservice testbed.
- Compare Redis vs local limiter using the same API.

Long-Term Objectives

- Extend the hybrid synchronization approach for global distributed rate limiting across data centers.

Related work

Rate limiting is an essential mechanism in large-scale distributed systems to ensure fair access and maintain backend stability. Common algorithms include the Fixed Window, Sliding Window, and **Token Bucket** models, each offering different trade-offs between accuracy, memory cost, and latency.

The **Token Bucket** method is widely adopted because it allows temporary bursts while maintaining an average request rate, which fits real-world E-commerce traffic patterns such as flash sales and checkout surges.

Many production systems implement these algorithms using **Redis**, which provides atomic operations and fast in-memory counters for centralized rate control. Some implementations employ **Redis Sorted Sets** to manage time-ordered request logs for sliding-window enforcement. However, centralized Redis limiters can introduce latency bottlenecks or single points of failure under heavy load. To overcome this, **distributed approaches** replicate local token buckets across multiple servers, reducing latency and improving scalability at the expense of strict global consistency.

Methodology

Setup

- 2–3 backend servers behind a simple load balancer (Docker Compose)
- Each exposes `/api` endpoint with either:
 - Centralized Redis limiter(shared counter)
 - Local limiter (in-memory token bucket)

Tools

- Language: Go
- Load testing: Locust
- Metrics: Latency (avg, 99th), allowed vs blocked count

Impact

This project demonstrates how different rate-limiting architectures affect performance, fairness, and resilience in real-world distributed systems. The results provide valuable insights for E-commerce developers, system architects, and API platform engineers who must handle unpredictable, large-scale user traffic while maintaining stability and customer satisfaction. By comparing centralized and distributed rate limiters under realistic workloads, our findings help stakeholders make data-driven design decisions about when to prioritize accuracy (for fairness and compliance) versus latency (for user experience and scalability).

Reference :

- [1] [Rate-limiting strategies and techniques:](#)
- [2] [Twitter rate limits](#)
- [3] [Better Rate Limiting With Redis Sorted Sets:](#)
- [4] [System Design — Rate limiter and Data modelling:](#)
- [5] [How we built rate limiting capable of scaling to millions of domains:](#)
- [6] [Redis website](#)